Around The World

App Store Launch Plan — ADHD-friendly step-by-step workload (2–3 week target)

> **Product:** Native iOS app + Vapor API (Bay Area pickup soccer)
> **How to use this PDF:** Do one step at a time. Only move on when you hit that step's *Done when* line. Check boxes as you go.
> **Golden rule:** Do not skip Auth / hosted API to "make screenshots." That kills the timeline.

v1.0 ships only: Sign in with Apple · Browse games (list + map) · Claim spot / waitlist · Host & publish · Edit profile · My Games · Settings (EN/ES) · Solid empty/error states.
Cut from v1.0: Push notifications · Email · Google venue photos (unless key ready day 1) · Friends graph · Perfect profile redesign · Marketing site polish.

0. Master checklist (print & tick)

```
[ ] 0.1  Launch Kit folder + secrets file
[ ] 0.2  Mac tools (Xcode, repo pulled)
[ ] 1.1  Apple Developer Program active
[ ] 1.2  App Store Connect app + Bundle ID created
[ ] 1.3  Supabase project + migrations + 4 secrets saved
[ ] 1.4  Privacy policy URL + support URL live
[ ] 2.1  Vapor API live on HTTPS (/health works on phone)
[ ] 2.2  iOS pointed at HTTPS API; runs on device
[ ] 3.1  Sign in with Apple capability in Xcode
[ ] 3.2  Apple provider configured in Supabase
[ ] 3.3  Auth acceptance checklist (all 6 checks) pass
[ ] 4.1  Host Publish works for signed-in user
[ ] 4.2  Claim Spot works
[ ] 4.3  Profile edit + save works
[ ] 4.4  Browse has Bay Area listings (seed or real)
[ ] 5.1  Archive uploaded to App Store Connect
[ ] 5.2  Internal TestFlight verified
[ ] 5.3  External TestFlight (5–10 testers)
[ ] 6.1  Store listing metadata filled
[ ] 6.2  Screenshots uploaded
[ ] 6.3  Privacy Nutrition Labels + policy URL
[ ] 6.4  App Review contact + notes
[ ] 7.1  Feature freeze + final build
[ ] 7.2  Submitted to App Review
[ ] 7.3  Released (Ready for Sale / Pending Release)
```

Critical path (order matters)

1. Apple Developer + App Store Connect app record

2. Supabase (Postgres + Auth + JWT secret)
3. Deploy Vapor on HTTPS → `DATABASE_URL`
4. iOS: Sign in with Apple + JWT on API + wire Claim / Host / Profile
5. Seed Bay Area listings (or staging `SEED_DEMO=1`)
6. TestFlight (internal → 5–10 external)
7. Privacy labels + screenshots + review notes
8. Submit 1.0

PHASE 0 Setup (1–2 hours)

Step 0.1 — Make a "Launch Kit" folder

What you're doing: One place for passwords, links, and checkboxes so nothing lives only in your head.

1. On your Mac, create a folder: `AroundTheWorld-Launch`
2. Inside it, create: `SECRETS-DO-NOT-COMMIT.txt`
3. Create: `CHECKLIST.txt` and paste the master checklist from above

> **Done when: Both files exist and you can open them without hunting.**

Step 0.2 — Confirm you have the tools

On your **Mac** (required for App Store):

1. Install / open **Xcode** from the Mac App Store (latest stable)
2. Open Xcode once → accept license → let components install
3. Pull the latest repo:

```
cd path/to/your/repo
git pull origin main
```

4. Confirm you can open: `ios/AroundTheWorld/AroundTheWorld.xcodeproj`

> **Done when: Xcode opens the project without error.**

PHASE 1 Accounts (Day 1 / weekend)

Step 1.1 — Apple Developer Program

What you're doing: Paying Apple so you can use TestFlight + App Store.

1. Go to `https://developer.apple.com/programs/`
2. Click **Enroll** / Start Your Enrollment
3. Sign in with the Apple ID that will own the account
4. Choose **Individual** (simplest) unless you already have a company D-U-N-S
5. Pay the annual fee and finish enrollment
6. Wait until status shows **Active** (hours; sometimes up to ~48h)

Save in secrets file: Apple ID email · Team name · Team ID (Account → Membership)

> **Done when: developer.apple.com/account shows Active membership, not Pending.**

> If stuck: Enrollment pending → wait. Do not start TestFlight until Active.

Step 1.2 — Create the app in App Store Connect

What you're doing: The listing shell Apple will review later.

1. Go to `https://appstoreconnect.apple.com` → **Apps** → **+** → **New App**
2. Platforms: **iOS**
3. Name: **Around The World** (or final unique name)
4. Primary Language: English (U.S.)
5. Register Bundle ID if needed:
   - `https://developer.apple.com/account/resources/identifiers/list`
   - **+** → App IDs → App
   - Explicit Bundle ID, e.g. `app.aroundtheworld.ios` — **never change this later**
   - Enable capability: **Sign In with Apple**
   - Register
6. Back in App Store Connect: select that Bundle ID
7. SKU: e.g. `ATW001` (internal only)
8. User Access: Full Access → Create

Save: Bundle ID · App name · App Apple ID number

> **Done when: You see the app page with App Store / TestFlight tabs.**

Step 1.3 — Create Supabase project

What you're doing: Hosted Postgres + Auth for real users.

1. Go to `https://supabase.com` → New project
2. Name: `around-the-world` · strong DB password · US West region preferred
3. Create → wait until healthy (green)
4. Project Settings → API — copy:
   - **Project URL** → `SUPABASE_URL`
   - **anon public** key → `SUPABASE_ANON_KEY`
   - **JWT Secret** → `SUPABASE_JWT_SECRET`
5. Project Settings → Database — copy URI → `DATABASE_URL` (replace password placeholder)
6. SQL Editor — run repo migrations **in order**:
   - `supabase/migrations/202607250001_auth_profiles.sql`
   - `supabase/migrations/202608080001_vapor_core_tables.sql`
   - `supabase/migrations/202608080002_profile_age.sql`

```
SUPABASE_URL=...
SUPABASE_ANON_KEY=...
SUPABASE_JWT_SECRET=...
DATABASE_URL=postgresql://...
DB_PASSWORD=...
```

Done when: Migrations ran; all four secrets are in your secrets file.

If a SQL statement errors because a table already exists, stop and note it — don't re-run files out of order.

Step 1.4 — Privacy policy + support URL

What you're doing: Two public https links Apple will require.

1. Create a public page (Notion / Carrd / GitHub Pages) titled **Privacy Policy — Around The World**
2. Include at minimum:
   - What you collect (Apple/Supabase account, profile fields, game participation, location when allowed)
   - Why (nearby games, run the service)
   - Location is optional / when-in-use
   - How to request deletion + contact email
3. Publish → copy https URL
4. Create Support URL (can be same site with a Support section) or a support email page

Done when: Both URLs open on your phone in Safari with no login wall.

PHASE 2 Host the API (Day 1–2)

Your iPhone **cannot** use `127.0.0.1` in production. You need a public HTTPS API.

Step 2.1 — Deploy Vapor (pick ONE host)

Recommendation: Railway (simpler UI) or Fly.io. Pick one; ignore the other.

Environment variables to set:

```
DATABASE_DRIVER=postgres
DATABASE_URL=<from secrets>
DATABASE_TLS=require
SEED_DEMO=1
SUPABASE_URL=<from secrets>
SUPABASE_JWT_SECRET=<from secrets>
PORT=8081
```

Railway path: railway.app → New Project → deploy Backend → set env vars → enable public HTTPS domain → open `https://YOUR-DOMAIN/health`

Fly path: install flyctl → login → deploy Backend → `fly secrets set ...` → check health URL

Save: `API_BASE_URL=https://YOUR-DOMAIN` (no trailing slash)

Done when: From your phone on cellular Safari, `/health` and `/api/v1/games` return data.

TLS/DB errors → check password URL-encoding in DATABASE_URL.
Empty games → keep SEED_DEMO=1 for staging.
Deploy rabbit-hole → switch hosts rather than burning a whole day.

Step 2.2 — Point iOS at the hosted API

1. Open `ios/AroundTheWorld/AroundTheWorld.xcodeproj`
2. Target → Build Settings → set `API_BASE_URL` to your HTTPS URL (Release + Debug for device testing)
3. Set `SUPABASE_URL` and `SUPABASE_ANON_KEY` for the client
4. Signing & Capabilities → select your **Team**
5. Run on a **physical iPhone** (not only Simulator)

Done when: Matches loads real games from HTTPS within ~8 seconds (not an infinite spinner).

PHASE 3 Auth (Day 2–5) — the big blocker

Without this, Claim Spot / My Games / Host / Profile stay empty. **Do not skip ahead.**

Step 3.1 — Enable Sign in with Apple (Apple side)

1. developer.apple.com → Identifiers → your App ID → enable **Sign In with Apple** → Save
2. Xcode → target → Signing & Capabilities → **+ Capability** → Sign In with Apple

> **Done when: Capability shows with no red error.**

Step 3.2 — Enable Apple provider in Supabase

1. Supabase → Authentication → Providers → Apple
2. Follow Supabase's current native iOS "Sign in with Apple" docs
3. Goal: app gets Apple credential → Supabase session JWT → Vapor accepts JWT

This is the main engineering chunk. Your job as owner: finish Phases 1–2 first so eng isn't blocked, then test Sign In on a real device.

> **Done when: You have a session JWT in the app and the backend accepts it.**

Step 3.3 — Auth acceptance checklist (must all pass)

Test on a physical iPhone against the hosted API:

1. Fresh install → **Sign in with Apple** succeeds
2. Open a game → **Claim Spot** → you appear in roster
3. **My Games** shows that game
4. **Host** → pin → publish → game appears in Browse
5. **Profile** edits age/skill/position/bio/city and saves
6. Kill app → reopen → still signed in

> **Done when: All 6 pass. If any fail, do not start external TestFlight.**

PHASE 4 Product wiring (alongside auth)

Step 4.1 — Host Publish

Current repo state: Location picker works; Publish is disabled pending `hostUserId`.

> **Done when: Signed-in user can publish; game shows in list/map with correct coordinates.**

Step 4.2 — Claim Spot

Current repo state: API call only runs if `currentUserID` is set.

> **Done when: Signed-in claim works; signed-out shows clear "Sign in to claim" (no crash).**

Step 4.3 — Profile

Functional form is enough for launch. Pretty redesign is optional.

> **Done when: Edit + reload after relaunch works.**

Step 4.4 — Listings

1. Staging: `SEED_DEMO=1` on hosted API, **or**
2. Create 10+ games via Host after auth

> **Done when: Browse is not empty for a new Bay Area tester.**

PHASE 5 TestFlight (Week 2)

Step 5.1 — Archive and upload

1. Xcode destination: **Any iOS Device (arm64)**
2. Product → Archive
3. Organizer → Distribute App → App Store Connect → Upload
4. Wait for processing (5–30+ min)
5. Answer export compliance honestly (typical HTTPS-only apps often use the exempt path)

> **Done when: Build appears under TestFlight ready to test.**

Step 5.2 — Internal testing

1. App Store Connect → TestFlight → Internal Testing
2. Add yourself + 1–2 trusted people
3. Install TestFlight app → install build
4. Re-run Step 3.3 checklist on the TestFlight build

> **Done when: You personally completed all 6 checks on a TestFlight build (not only Xcode debug).**

Step 5.3 — External testing (small)

1. Create External group: “Bay Area Beta”
2. Add 5–10 emails
3. First external build may need short Beta App Review
4. What to Test: “Bay Area pickup soccer. Sign in with Apple, browse, claim a spot, host a session.”

> **Done when: ≥3 external testers installed; bugs collected in one note.**

PHASE 6 Store listing (Week 2–3)

Step 6.1 — Metadata

- **Subtitle** (30 chars): e.g. Bay Area pickup soccer
- **Description:** short paragraph + bullets (browse, map, claim, host, profile)
- **Keywords:** comma-separated, 100 char limit
- **Category:** Sports
- Age / content questionnaire: answer honestly (bios = user-generated content)

Step 6.2 — Screenshots

Required sizes: **6.7"** and **6.1"** iPhone. Capture at least:

1. Browse / Matches
2. Map
3. Game detail with Claim
4. Host flow
5. Profile

Clean in-app screenshots are fine — no fancy marketing frames required.

Step 6.3 — Privacy Nutrition Labels

App Store Connect → App Privacy. Typically declare (only if true for your build):

- **Location** — Precise, when in use — App Functionality
- **Contact Info** — if you store name/email from Apple/profile
- **User Content** — bio / game details
- **Identifiers** — user ID for account

Paste Privacy Policy URL. Be accurate — don't claim data you don't collect.

Step 6.4 — App Review information

- Contact phone + email you will answer within 24h
- Notes: how to Sign in with Apple; that seed/Bay Area data exists; location optional for map
- Backend URL for reviewers if helpful

Done when: No missing-metadata warnings on the version page.

PHASE 7 Submit (Week 3)

Step 7.1 — Feature freeze

1. No new features 24h before submit
2. Only crash / blocker fixes
3. Bump build number (`CFBundleVersion`); version can stay 1.0
4. Upload final build → select it for version 1.0

Step 7.2 — Submit for Review

1. Add build to version
2. Answer export / advertising / IDFA questions (usually no tracking if you added no ads SDKs)
3. Add for Review → Submit to App Review

Step 7.3 — While waiting

- Keep API up
- Don’t rotate JWT secrets / DB passwords mid-review
- Watch email for Rejection — common: login broken, privacy mismatch, incomplete metadata

> **Done when: Status is Pending Developer Release or Ready for Sale — then release.**

Day-by-day schedule (ADHD one-job days)

Day	One job only
1	Apple Developer + ASC app + Bundle ID
2	Supabase + secrets file + SQL migrations
3	Deploy API → phone can hit /health
4–6	Auth + Claim + Host + Profile (Step 3.3 checklist)
7	Buffer — fix auth bugs only
8–9	TestFlight internal
10–12	External beta + fix P0 bugs
13–14	Screenshots, privacy labels, listing text
15–16	Submit 1.0
17–21	App Review response buffer

If a day fails, **repeat that day**. Do not skip lanes.

Risks that kill the timeline

Risk	Mitigation
Auth left to “week 2”	Start Sign in with Apple on day 1 of coding
Only localhost API	Deploy before any TestFlight
Waiting on perfect logo/profile	Ship good-enough icon + functional profile
Scope creep (push, chat, payments)	Explicit v1.1 list — not now
Review rejection	Working login + publish + accurate privacy
Empty browse	Seed staging DB before external testers

Where the project already stands

Area	Status
Browse / game detail / map	Working against API
Host location search + pin	UI ready; Publish not wired
Claim Spot / My Games / Profile	APIs exist; empty without signed-in user
Auth	Not built (Phase 2 in docs)
Hosted backend	Local SQLite only until you deploy
Secrets	Placeholders — you must supply
Push / email	Out of scope for launch
Web app	Dev/prototype — not the App Store product

Do this in the next 60 minutes

1. Create the `AroundTheWorld-Launch` folder + secrets file
2. Start **Step 1.1** Apple enrollment if not Active
3. If Apple is already Active, do **Step 1.2** Bundle ID + App Store Connect app
4. Stop. Don’t open Xcode for features until secrets + ASC exist

---